

Mathematical Formulation of Attention-Guided Reinforcement Learning

AttentionGuidedRL Repository

August 21, 2025

Abstract

This document provides a mathematical formulation of the Attention-Guided Reinforcement Learning system, where a language model (Llama-3.2-3B or GPT-2) learns to autonomously sequence its own training data. The key innovation is a **chain rule policy gradient** approach that combines:

1. **Direct reward optimization:** $\nabla_{\theta} r_{\theta}^t$ - optimizing the reward function itself
2. **Trajectory-average policy gradients:** $\bar{R}(\tau) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ - uniform credit assignment

The model generates vector queries using attention mechanisms and selects key-value pairs from Wikipedia articles to maximize learning progress. Unlike standard REINFORCE [1], this approach optimizes both the policy and the reward function simultaneously through differentiable reward computation, creating a more sophisticated learning dynamic.

Contents

1	Problem Formulation	1
1.1	State Space and Action Space	1
1.2	Trajectory and Episode Structure	1
2	Policy Definition	2
2.1	Vector Query Generation	2
2.2	Multi-Head Attention Similarity	2
3	Reward Function	3
4	Optimization Objective	3
5	Training Algorithm	4
6	Implementation Details	4
6.1	Model Architecture	4
6.2	Training Configuration	5
6.3	Data Processing	5
7	Mathematical Properties	5
8	Conclusion	5

1 Problem Formulation

1.1 State Space and Action Space

Definition 1 (State Space). *At timestep t , the state s_t consists of:*

$$s_t = (c_t, \mathcal{K}_t^{available}) \quad (1)$$

where:

- $c_t \in \mathbb{Z}^{L_t}$ is the context sequence of length L_t
- $\mathcal{K}_t^{available} \subseteq \{1, 2, \dots, K\}$ is the set of available key indices

Definition 2 (Action Space). *The action space $\mathcal{A}_t$ at timestep t is the set of available key indices:*

$$\mathcal{A}_t = \mathcal{K}_t^{available} \quad (2)$$

1.2 Trajectory and Episode Structure

A trajectory τ consists of T steps:

$$\tau = \{(s_1, a_1, r_1), (s_2, a_2, r_2), \dots, (s_T, a_T, r_T)\} \quad (3)$$

where T is the number of key-value pairs per episode (NUM_KV_PAIRS in the code).

2 Policy Definition

2.1 Vector Query Generation

The policy π_θ operates through a vector query mechanism parameterized by LoRA adapter weights θ [3].

Definition 3 (Query Vector Generation). *Given context c_t , the query vector is generated as:*

$$q_t = \text{Embed}_\theta(c_t \oplus [\text{“Query”}])_{-1} \quad (4)$$

where:

- $\text{Embed}_\theta(\cdot)$ extracts embeddings from the second-to-last attention layer
- $\oplus$ denotes sequence concatenation
- $(\cdot)_{-1}$ extracts the embedding of the last token (the “Query” token)
- $q_t \in \mathbb{R}^{d_{model}}$ where d_{model} is the model dimension

This focused extraction ensures that the model learns to encode its query intent specifically into the “Query” token, rather than diluting the signal across the entire context.

2.2 Multi-Head Attention Similarity

The system handles both standard Multi-Head Attention (MHA) and Grouped Query Attention (GQA) [2].

Definition 4 (Head-wise Similarity Computation). *For each attention head $h \in \{1, 2, \dots, H\}$ and key k :*

$$sim_{t,k}^{(h)} = \frac{(q_t^{(h)})^T k_k^{(group(h))}}{\sqrt{d_{head}}} \cdot \frac{1}{\tau} \quad (5)$$

where:

- $q_t^{(h)} \in \mathbb{R}^{d_{head}}$ is the query for head h
- $k_k^{(group(h))} \in \mathbb{R}^{d_{head}}$ is the key for the corresponding group
- $group(h) = \lfloor h \cdot G/H \rfloor$ maps heads to groups (for GQA)
- G is the number of key-value groups
- H is the number of query heads
- $d_{head} = d_{model}/H$ is the head dimension
- $\tau = 1.0$ is the temperature parameter

Definition 5 (Per-Head Probability Distribution). *For each head h , the probability distribution over keys is:*

$$p_{t,k}^{(h)} = \frac{\exp(sim_{t,k}^{(h)})}{\sum_{k' \in \mathcal{K}_t^{available}} \exp(sim_{t,k'}^{(h)})} \quad (6)$$

Definition 6 (Policy Distribution). *The final policy distribution is obtained by averaging over heads:*

$$\pi_\theta(k|s_t) = \frac{1}{H} \sum_{h=1}^H p_{t,k}^{(h)} \quad (7)$$

This preserves the probability distribution property: $\sum_{k \in \mathcal{K}_t^{available}} \pi_\theta(k|s_t) = 1$.

3 Reward Function

Definition 7 (Step Reward). *The reward at step t is the per-token average log probability:*

$$r_\theta^t = \frac{1}{|v_t|} \sum_{i=1}^{|v_t|} \log p_\theta(v_{t,i} | c_t, k_t, v_{t,<i}) \quad (8)$$

where:

- v_t are the value tokens at step t

- k_t are the selected key tokens at step t
- $p_\theta(\cdot)$ is the current adapter model probability
- $v_{t,i}$ is the i -th token and $v_{t,<i}$ represents preceding tokens

Configuration Option: The system supports two reward modes:

- `subtract_base_model_logprobs=False` (default): Uses the formula above
- `subtract_base_model_logprobs=True`: Subtracts reference model log probabilities:

$$r_\theta^t = \frac{1}{|v_t|} \sum_{i=1}^{|v_t|} \left(\log p_\theta(v_{t,i} \mid \cdot) - \log p_{\text{ref}}(v_{t,i} \mid \cdot) \right) \quad (9)$$

4 Optimization Objective

Theorem 1 (Trajectory-Average Policy Gradient). *The system maximizes:*

$$\mathcal{J}(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [\bar{R}(\tau)] \quad (10)$$

where $\bar{R}(\tau) = \frac{1}{T} \sum_{t=1}^T r_\theta^t$ is the trajectory average reward.

Chain Rule Gradient Derivation: Since both policy and rewards depend on θ , the gradient is:

$$\nabla_\theta \mathcal{J}(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\frac{1}{T} \sum_{t=1}^T \nabla_\theta r_\theta^t + \bar{R}(\tau) \sum_{t=1}^T \nabla_\theta \log \pi_\theta(a_t | s_t) \right] \quad (11)$$

This gives two terms:

1. **Reward gradient term:** $\frac{1}{T} \sum_{t=1}^T \nabla_\theta r_\theta^t$ - direct optimization of reward function
2. **Policy gradient term:** $\bar{R}(\tau) \sum_{t=1}^T \nabla_\theta \log \pi_\theta(a_t | s_t)$ - trajectory-average weighted policy gradient

Each action receives uniform credit: the trajectory average reward $\bar{R}(\tau)$.

5 Training Algorithm

Loss Function: The training implements the full chain rule gradient:

$$\mathcal{L}(\theta) = \mathbb{E}_{\text{batch}} \left[-\frac{1}{T} \sum_{t=1}^T r_\theta^t - \text{sg}[\bar{R}(\tau)] \sum_{t=1}^T \log \pi_\theta(a_t | s_t) \right] \quad (12)$$

where:

- $\bar{R}(\tau) = \frac{1}{T} \sum_{t=1}^T r_\theta^t$ is the trajectory average reward
- The first term optimizes rewards directly via $\nabla_\theta r_\theta^t$

Algorithm 1 Chain Rule Policy Gradient Training

Require: Base model, tokenizer, Wikipedia dataset

- 1: Initialize LoRA adapter parameters θ
 - 2: **for** each training episode **do**
 - 3: Sample Wikipedia article and extract key-value pairs
 - 4: Generate trajectory τ using current policy π_θ
 - 5: Compute trajectory average reward $\bar{R}(\tau)$
 - 6: Compute loss and update parameters via gradient descent
 - 7: **end for**
-

- The second term is the trajectory-average weighted policy gradient

Key Properties:

- Each action receives uniform credit: the trajectory average reward
- No baseline subtraction or value function needed
- Simple implementation with uniform credit assignment

6 Implementation Details

6.1 Model Architecture

- **Base Models:** Llama-3.2-3B or GPT-2
- **LoRA Configuration:** rank=8, $\alpha=16$, dropout=0.0
- **Precision:** bfloat16 for computation, float32 for similarity calculations
- **Context Window:** Up to model maximum (2048 for GPT-2, 4096+ for Llama)

6.2 Training Configuration

- **Optimizer:** AdamW with learning rate 5×10^{-4}
- **Batch Size:** 4 trajectories per update
- **Gradient Clipping:** Norm clipped to 1.0
- **Episodes:** 10,000 total training episodes
- **Checkpoint Frequency:** Every 100 episodes

6.3 Data Processing

- **Dataset:** Wikipedia 2022-03-01 English subset
- **Token Counts:** 10 tokens per key, 10 tokens per value
- **Trajectory Length:** Up to 10 key-value pairs per episode
- **Prefixes:** "Key: " and "Value: " for context building

7 Mathematical Properties

The trajectory-average approach gives uniform credit assignment where each action receives the same weight: the trajectory average reward $\bar{R}(\tau)$. This treats all actions in a trajectory equally, following standard REINFORCE convergence guarantees [1].

8 Conclusion

This mathematical formulation describes a reinforcement learning system where a language model learns to sequence its own training data using:

1. **Chain rule policy gradients:** Simultaneous optimization of both policy and reward function
2. **Vector-based policy:** Query generation from attention embeddings
3. **Multi-head attention similarity:** For key selection across MHA/GQA architectures
4. **Trajectory-average credit assignment:** Uniform weighting across all actions
5. **Differentiable reward computation:** Enabling direct reward optimization

The key innovation is the **chain rule approach** that goes beyond standard REINFORCE [1] by optimizing the reward function directly ($\nabla_{\theta} r_{\theta}^t$) while using trajectory-average rewards for policy gradients ($\text{sg}[\bar{R}(\tau)] \nabla_{\theta} \log \pi_{\theta}$). This creates a sophisticated learning dynamic where the model simultaneously improves its policy and the rewards it receives.

References

- [1] Williams, R. J. (1992). *Simple statistical gradient-following algorithms for connectionist reinforcement learning*. Machine Learning, 8(3-4), 229-256.
- [2] Shazeer, N. (2019). *Fast transformer decoding: One write-head is all you need*. arXiv preprint arXiv:1911.02150.
- [3] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., ... & Chen, W. (2021). *LoRA: Low-rank adaptation of large language models*. arXiv preprint arXiv:2106.09685.